Dos bases más y una caché

Lab S11 - Sistemas Distribuidos 2027-1
Oscar Montiel | Facultad de Ingeniería, UNAM
Ejecución: 2 de noviembre de 2026, Kubuntu y Docker Compose.

Partí de la rama **06-poliglota-base**. PostgreSQL conserva cuentas, dinero, movimientos y Outbox; Redis guarda una copia del saldo para mostrar; MongoDB guarda los avisos de transferencias. Las capturas son de las terminales donde ejecuté los comandos y consulté sus resultados.

Parte 0. El punto de partida

Abrí las cuentas de Ana (1000), Beto (500) y Carla (300), y transferí 200 de Ana a Beto. Antes de habilitar la caché, diez GET provocaron **diez lecturas nuevas a PostgreSQL**; leidaEn cambió en cada respuesta. Redis tuvo cero hits y misses; Mongo tenía cero avisos. El aviso inicial seguía en la bandeja de PostgreSQL.

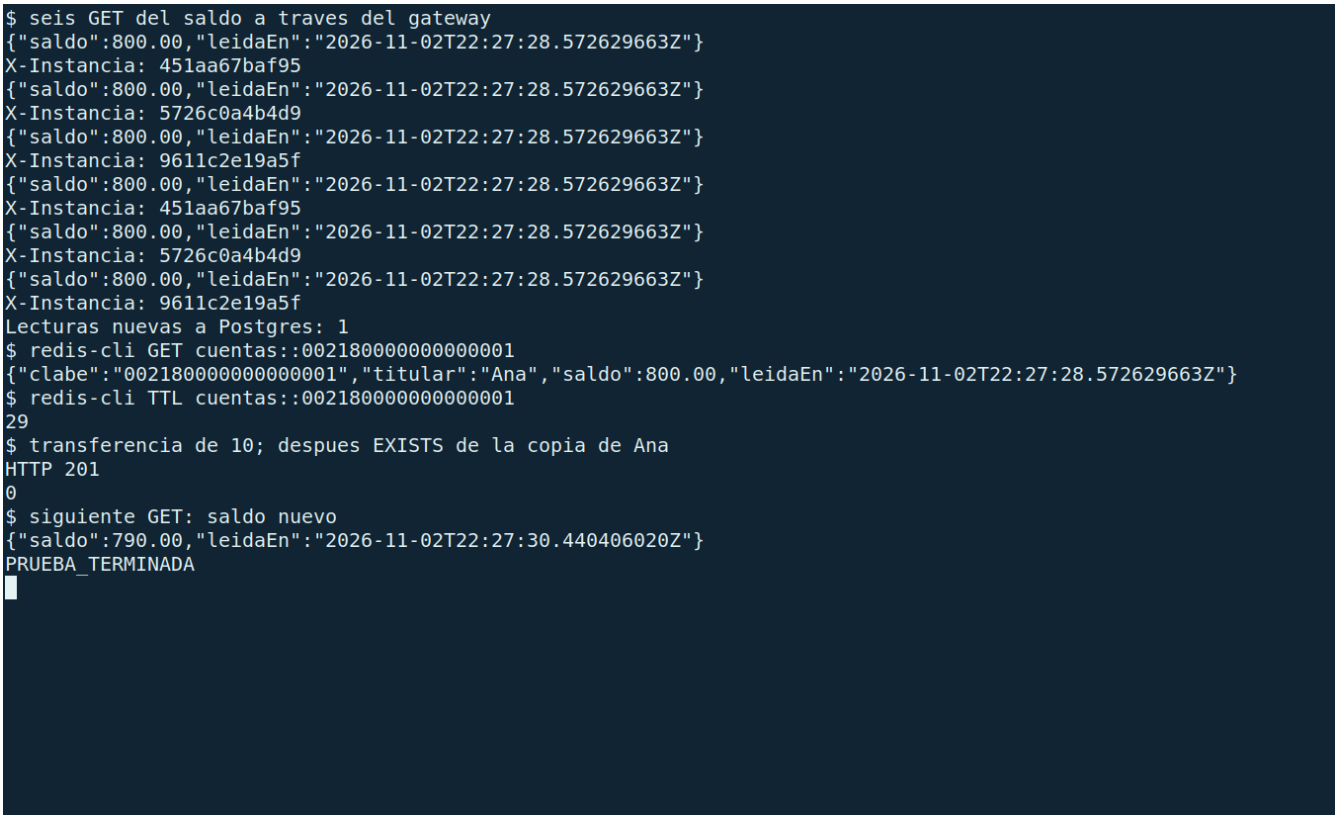
```
$ docker compose exec redis redis-cli ping
PONG
$ mongosh: avisos en Mongo antes del cambio
0
$ diez GET del saldo, sin cache
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.327177899Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.353758684Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.381285928Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.409254301Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.437309880Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.462104438Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.491855436Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.521837936Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.545427882Z"}
{"saldo":800.00,"leidaEn":"2026-11-02T22:25:55.568050192Z"}
Lecturas nuevas a Postgres: 10
$ redis-cli INFO stats
keyspace_hits:0
keyspace_misses:0
$ bandeja vieja de Postgres: avisos de Ana
1
PRUEBA_TERMINADA
```

Captura de pantalla. Diez consultas, diez lecturas y ningún uso de Redis.

Parte 1. Una caché compartida para mostrar el saldo

Agregué **@EnableCaching**, un TTL de 30 s y un serializador JSON de CuentaVista en ConfiguracionCache. **@Cacheable** se aplica únicamente a ConsultaDeSaldo.consultar: en un miss lee PostgreSQL y deja una copia; en un hit devuelve esa copia. CuentaService.consultar, usado para autorizar cargos, sigue leyendo PostgreSQL.

Las seis respuestas conservaron el mismo leidaEn y pasaron por **tres X-Instancia distintos**: solo hubo una lectura nueva a PostgreSQL. Después de transferir 10, EXISTS dio 0 y el GET siguiente mostró 790. El borrado se hace con @CacheEvict en cargar y abonar; transactionAware() lo difiere hasta el commit si hay una transacción activa.

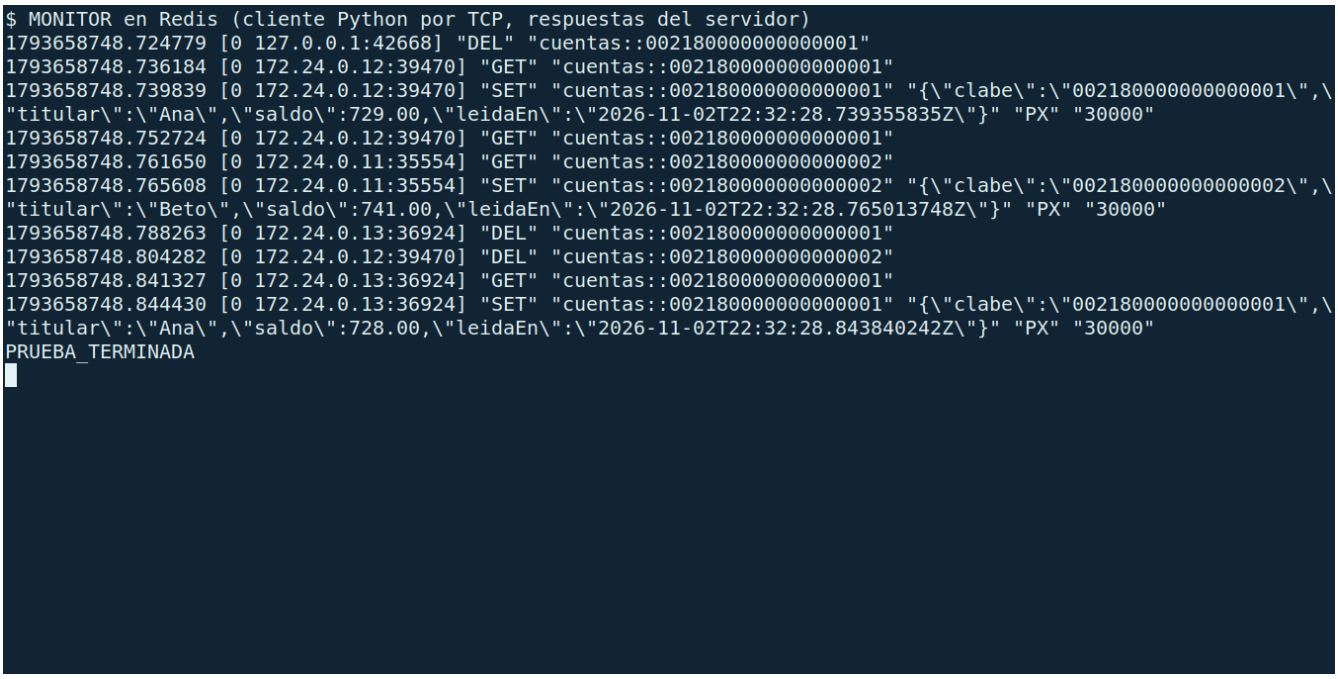


Captura de pantalla. Las tres réplicas comparten JSON, TTL y la invalidación de la misma clave.

Parte 1. Observar GET, SET y DEL

Observé el comando MONITOR del servidor desde otra terminal. Para evitar que el cliente de Compose quedara abierto al terminar, usé un cliente pequeño en Python que recibe las respuestas del protocolo Redis por TCP durante doce segundos. La salida viene del servidor, no de datos simulados.

En esta prueba adicional de un peso aparecen las comprobaciones de la saga, los **DEL de origen y destino** y después el GET que falla en caché y el SET con el saldo nuevo, 728, y **PX 30000**. Ese SET pertenece a la lectura posterior, no al método que carga el dinero.



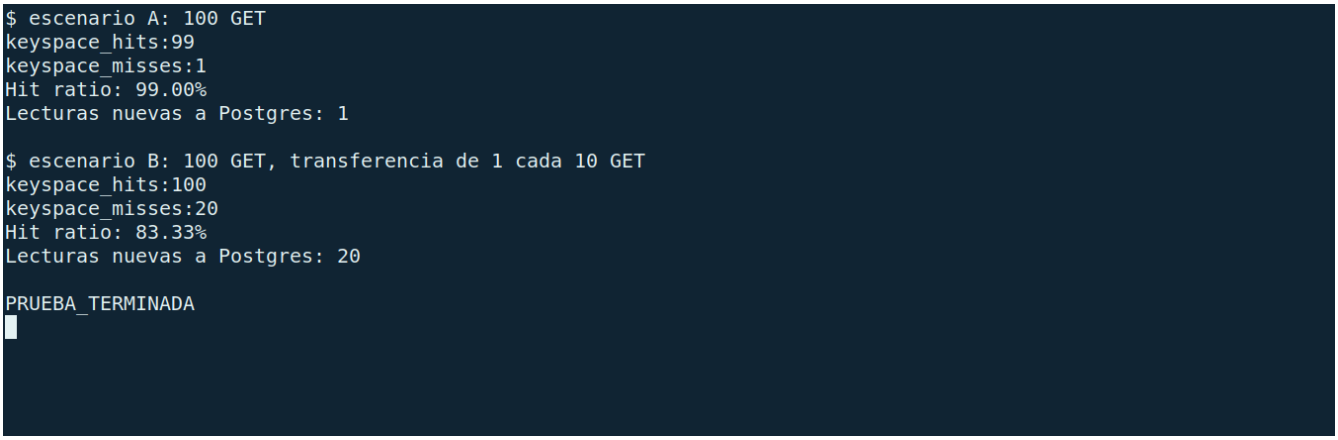
Captura de pantalla. Comandos recibidos por Redis durante una transferencia y el GET posterior.

Medición 1. Hit ratio

Antes de cada escenario borré solo la clave de Ana y reinicié los contadores con CONFIG RESETSTAT. No ejecuté redis-cli GET dentro de la medición. Conté las lecturas nuevas comparando el log de las tres réplicas antes y después.

Escenario	Hits / misses	Hit ratio	Lecturas PG
A: 100 GET	99 / 1	99.00 %	1
B: 100 GET + 10 transferencias	100 / 20	83.33 %	20

En B, cada transferencia borra las copias de Ana y Beto. Además, la saga hace un GET de cada cuenta antes del cargo: los contadores incluyen **120 accesos a caché**, no solo los cien GET manuales. Por eso los siguientes accesos necesitan volver a PostgreSQL y baja el porcentaje. La fórmula es hits / (hits + misses).



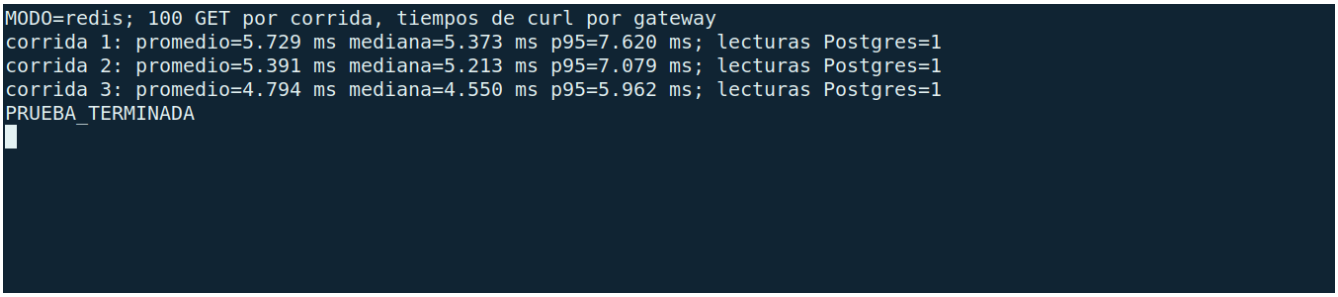
Captura de pantalla. Resultados medidos de A y B.

Medición 2. Latencia de cien consultas por corrida

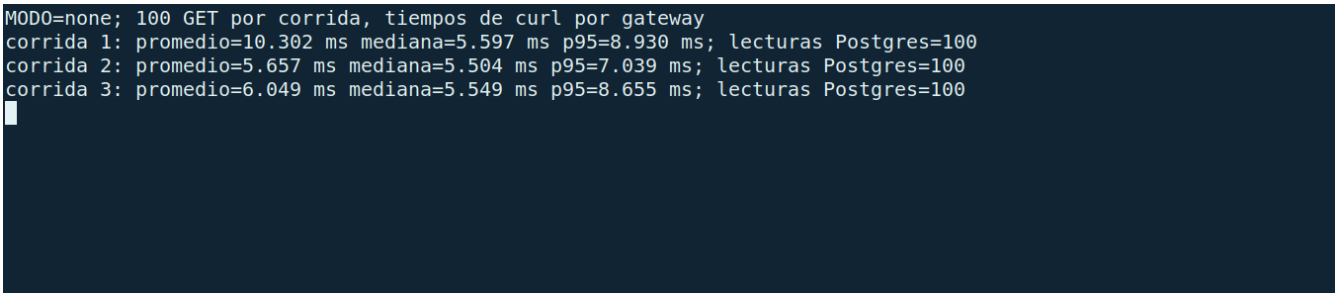
Corrí tres rondas con caché y tres con CACHE_TIPO=none. Cada ronda guardó cien tiempos de curl por el gateway. Con caché empecé borrando la clave de Ana: una lectura fue miss y las otras 99 fueron hits. Los conteos son del mensaje de log de ConsultaDeSaldo; no cuentan las lecturas internas de autorización.

Modo / corrida	Promedio ms	Mediana ms	p95 ms	Lecturas PG
Redis / 1	5.729	5.373	7.620	1
Redis / 2	5.391	5.213	7.079	1
Redis / 3	4.794	4.550	5.962	1
Sin caché / 1	10.302	5.597	8.930	100
Sin caché / 2	5.657	5.504	7.039	100
Sin caché / 3	6.049	5.549	8.655	100

La ganancia más clara fue de **capacidad**: tres lecturas a PostgreSQL en 300 consultas con caché frente a 300 sin ella. La mediana mejoró poco. Son pruebas locales con una tabla pequeña, corridas secuenciales y sin calentamiento de JVM controlado; el promedio de la primera ronda sin caché tuvo un valor alto y no lo descarté. Estos números no predicen una carga de producción.



Captura de pantalla. Las tres corridas con caché.



Captura de pantalla. Las tres corridas sin caché.

Parte 2. Un documento por aviso en MongoDB

Completé documento(), registrar(), crearIndice() y ultimos() en BitacoraDeAvisos. El consumidor ahora llama **solo a bitacora.registrar** para los avisos de transferencias y hace ack después de guardar. El endpoint antiguo de PostgreSQL queda para SPEI, como en la base del lab.

Ana recibe SMS, Beto correo y Carla push. Los documentos comparten evento, CLABE, tipo, monto, saldo y fecha; cada canal añade sus campos. Usé **Decimal128** para el dinero y la fecha del evento. Esta consulta proyecta los campos propios de cada canal y comprueba los tipos de los importes.

```
$ transferencia Ana -> Beto por 20, y Ana -> Carla por 30
HTTP 201
HTTP 201
$ mongosh: un documento de cada canal, sin campos comunes repetidos
{
  canal: 'SMS',
  telefono: '+52 55 5555 0101',
  texto: 'TRANSFERENCIA_ENVIADA por 20, saldo resultante 759.00'
}
{
  canal: 'CORREO',
  correo: 'beto@correo.mx',
  asunto: 'Recibiste una transferencia',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 20, saldo resultante 741.00'
}
{
  canal: 'PUSH',
  token: 'fcm:demo-002180000000000003',
  titulo: 'Recibiste una transferencia',
  cuerpo: 'TRANSFERENCIA_RECIBIDA por 30, saldo resultante 330.00'
}
$ tipos de monto y saldo: Decimal128
[
  {
    monto: 'decimal',
    saldo: 'decimal'
  }
]
$ avisos con telefono (esquema al leer)
{"clabe":"002180000000000001","canal":"SMS","telefono":"+52 55 5555 0101"}
{"clabe":"002180000000000001","canal":"SMS","telefono":"+52 55 5555 0101"}
PRUEBA_TERMINADA
```

Captura de pantalla. Transferencias a Beto y Carla, tres formas y dinero decimal.

Parte 2. Últimos diez avisos y uso del índice

Hice doce transferencias de un peso a Beto. El endpoint devolvió **diez avisos CORREO ordenados por fecha descendente**. Creé clabe_fecha con clabe ascendente y fecha descendente, porque esa es la forma de la consulta: igualdad de CLABE, orden temporal y límite de diez.

En el explain con índice apareció **IXSCAN** y se examinaron diez documentos. Con hint natural aparecieron **COLLSCAN y SORT**, y se examinaron los treinta documentos que había en ese momento. Son dos planes para la misma consulta y los dos devuelven diez; cambia el trabajo que hacen.

```
$ doce transferencias de 1 a Beto
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
HTTP 201
$ GET /api/notificaciones/002180000000000002 (resumen comprobado)
{
  "cantidad": 10,
  "canales": [
    "CORREO"
  ],
  "orden_descendente": true,
  "primera": "2026-11-02T22:32:58.277+00:00",
  "ultima": "2026-11-02T22:32:57.693+00:00"
}
$ mongosh: indices
{"v":2,"key":{"_id":1},"name":"_id_"}
{"v":2,"key":{"clabe":1,"fecha":-1},"name":"clabe_fecha"}
$ explain con indice y con hint natural
{"natural":false,"plan":{"stage":"LIMIT","limitAmount":10,"inputStage":{"stage":"FETCH","inputStage":{"stage":"I
XSCAN","keyPattern":{"clabe":1,"fecha":-1},"indexName":"clabe_fecha","isMultiKey":false,"multiKeyPaths":{"clabe"
:[],"fecha":[]},"isUnique":false,"isSparse":false,"isPartial":false,"indexVersion":2,"direction":"forward","inde
xBounds":{"clabe":["[\"002180000000000002\", \"002180000000000002\"]"],"fecha":["[MaxKey, MinKey]"]}}}},"devuelt
os":10,"documentos":10}
{"natural":true,"plan":{"stage":"SORT","sortPattern":{"fecha":-1},"memLimit":104857600,"limitAmount":10,"type":"
simple","inputStage":{"stage":"COLLSCAN","filter":{"clabe":{"$eq":"002180000000000002"}},"direction":"forward"}}
,"devueltos":10,"documentos":30}
PRUEBA_TERMINADA
```

Captura de pantalla. Límite, orden, índices y los dos planes de ejecución.

Parte 3. El evento duplicado

Publiqué dos veces el evento fijo de la guía usando la API de RabbitMQ y CLABEs de prueba ...091 y ...092. El resultado fue **dos avisos, no cuatro**. Los dos publishes devolvieron routed:true y el log identificó los dos documentos que ya estaban.

```
$ publicar el mismo evento DOS veces (mismo eventoId y destinatarios)
{"routed":true}
{"routed":true}
$ mongosh: cantidad de avisos de ese evento
2
$ log: avisos que ya estaban
[avisos] 11111111-2222-4333-8444-555555555555:002180000000000091 ya estaba: evento repetido, no se duplica
[avisos] 11111111-2222-4333-8444-555555555555:002180000000000092 ya estaba: evento repetido, no se duplica
PRUEBA_TERMINADA
```

Captura de pantalla. El segundo procesamiento no crea avisos nuevos.

El _id es **eventoId:clabe**. No basta eventoId porque una transferencia genera dos destinatarios. Mongo rechaza una segunda inserción del mismo _id. Se inserta uno por uno: si el proceso cae después del primero, al reentregar se ignora ese duplicado y todavía se puede guardar el segundo. No hace falta escribir otra marca en PostgreSQL.

Parte 4. La pantalla puede mostrar un saldo viejo

Guardé en caché el saldo 716 y cambié PostgreSQL a 5 directamente, sin pasar por @CacheEvict. El dato viejo seguía en pantalla a los 26 s; la primera lectura fresca observada fue a los **31 s**. Como muestreé cada cinco segundos, no afirmo que la entrada duró exactamente 31 s: el TTL era de 30 s y el cambio quedó entre esas dos observaciones. leidaEn permaneció fijo hasta la lectura nueva.

```
$ lleno Redis con el saldo actual
{"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
$ UPDATE cuenta SET saldo=5 por fuera de la aplicacion
UPDATE 1
$ cada 5 s: tiempo transcurrido, TTL, saldo y leidaEn
t= 1s TTL= 30 {"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
t= 6s TTL= 25 {"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
t=11s TTL= 20 {"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
t=16s TTL= 14 {"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
t=21s TTL=  9 {"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
t=26s TTL=  4 {"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
t=31s TTL= -2 {"saldo":5.00,"leidaEn":"2026-11-02T22:33:46.677025651Z"}
t=36s TTL= 25 {"saldo":5.00,"leidaEn":"2026-11-02T22:33:46.677025651Z"}
$ restauro el saldo de Ana: 716.00 y borro la copia de prueba
{"saldo":716.00,"leidaEn":"2026-11-02T22:33:56.993429082Z"}
PRUEBA_TERMINADA
```

Captura de pantalla. TTL decreciente, lectura nueva y restauración del saldo.

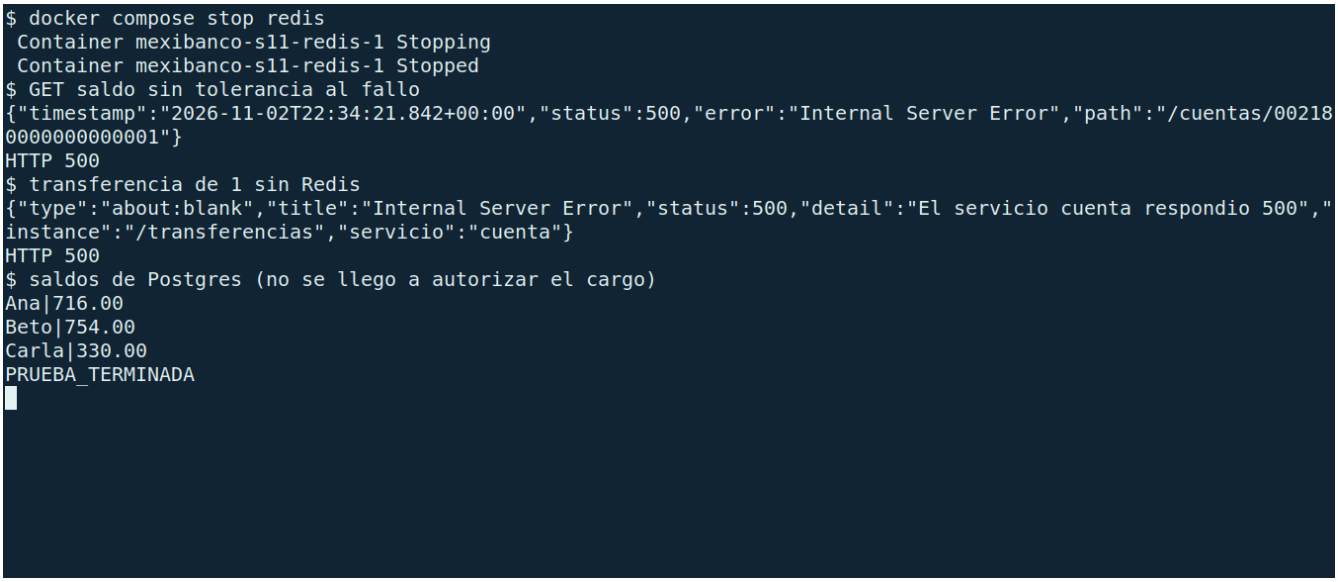
Mientras la pantalla decía 716, intenté transferir 100 en otra terminal. Recibí **HTTP 422: saldo insuficiente**; el cargo leyó los 5 de PostgreSQL. Al terminar restauré los 716 y borré la copia de prueba.

```
$ GET: lo que ve Ana mientras Redis tiene el saldo viejo
{"saldo":716.00,"leidaEn":"2026-11-02T22:33:15.822945934Z"}
$ Postgres: la verdad del saldo
5.00
$ intento transferir 100 a Beto
{"type":"about:blank","title":"Unprocessable Entity","status":422,"detail":"Saldo insuficiente en la CLABE 00218
0000000000001","instance":"/transferencias","servicio":"cuenta"}
HTTP 422
PRUEBA_TERMINADA
```

Captura de pantalla. El saldo viejo no autoriza gastar dinero.

Reto. Qué pasó cuando Redis se apagó

Primero probé la versión con caché normal. Al detener Redis, el GET devolvió **500** y la transferencia también devolvió **500** durante su comprobación inicial. Los saldos no cambiaron en esa prueba. La copia para mostrar se había convertido en una dependencia que podía impedir operar.



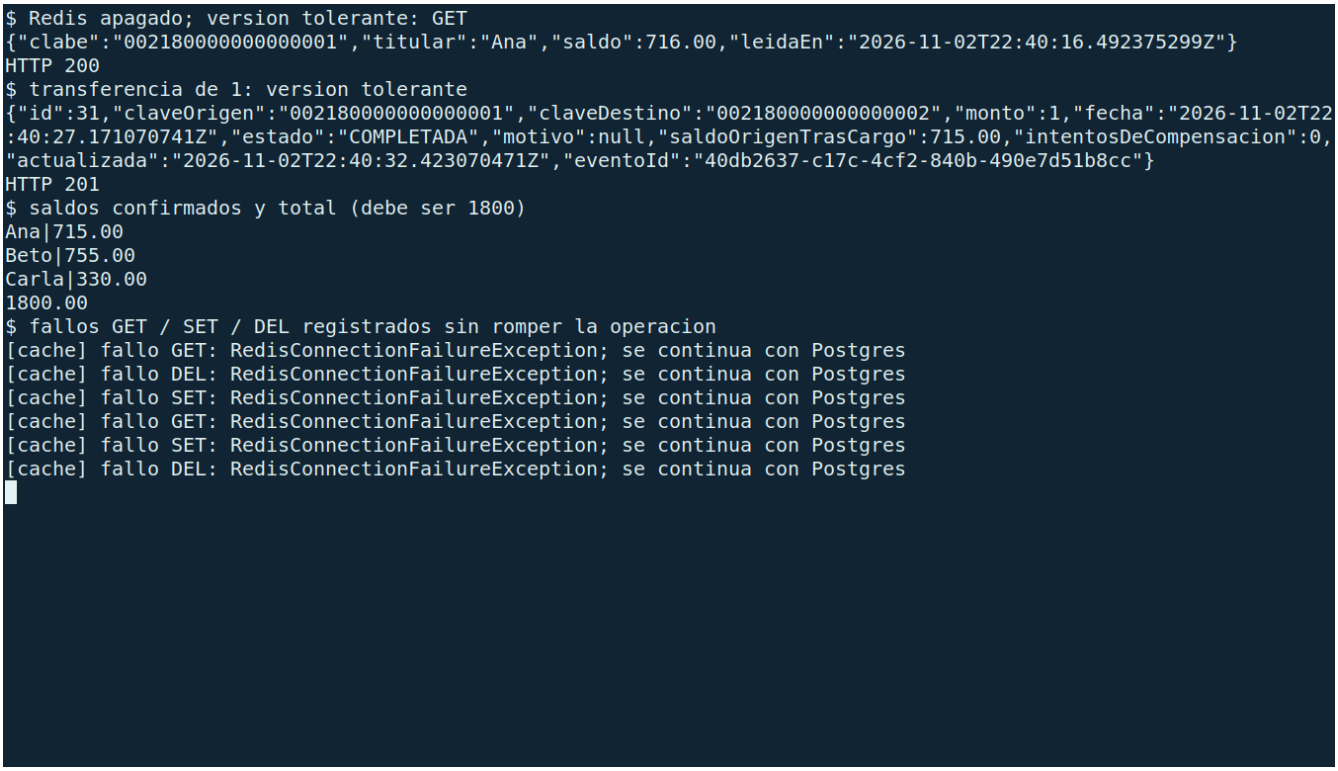
Captura de pantalla. Fallo observado antes de aplicar la tolerancia.

Un CacheErrorHandler por sí solo no cubre todas las invalidaciones diferidas: transactionAware() puede ejecutar el DEL en el callback posterior al commit, fuera del interceptor que maneja ese error. Por eso envolví **RedisCacheWriter**, que ejecuta la operación real, y atrapé únicamente errores de acceso a la caché. GET fallido se vuelve miss; SET y DEL fallidos se registran y se omiten.

Reto. Redis apagado y dinero consistente

El primer intento del escritor tolerante mostró otro riesgo real: el cargo se confirmó pero el DEL fallido tardó más que los 2 s de espera de la saga. Ese intento dejó un peso descontado y una saga FALLIDA. Restituí ese peso en la base aislada antes de repetir; no lo conté como éxito. Ajusté la espera HTTP a 10 s, conservando límites de Redis de 500 ms; la resolución de su nombre también puede agregar espera.

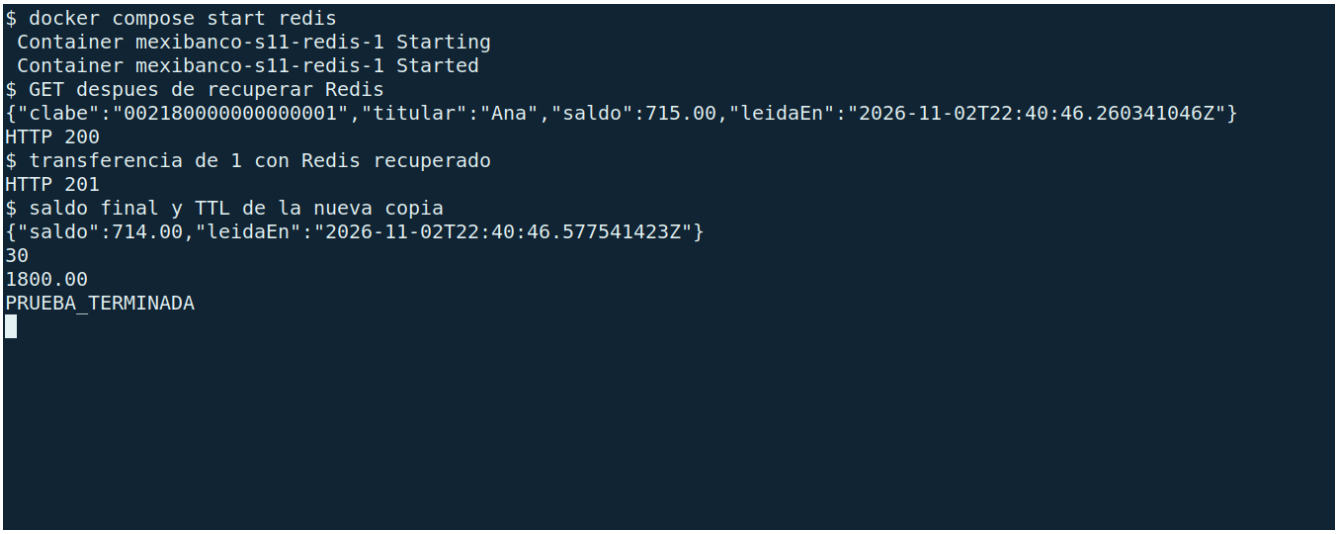
Con la configuración final y Redis aún apagado, el GET respondió **200**, la transferencia respondió **201** y la suma de saldos fue **1800**. El log muestra los fallos de GET, SET y DEL que se omitieron. Acepté menor rendimiento durante la caída para conservar la operación y poder comprobar el resultado.



Captura de pantalla. Prueba final del reto: respuesta y saldos confirmados.

Reto. Recuperación y límites de la solución

Encendí Redis, hice otra transferencia y volví a consultar el saldo. La respuesta fue correcta, se creó una copia con TTL y el total permaneció en 1800. Redis no tiene persistencia en este Compose: al parar y arrancar el proceso perdió sus copias anteriores y se llenó de nuevo desde PostgreSQL.



Captura de pantalla. Recuperación del servicio y nueva copia de saldo.

Omitir un DEL fallido no vuelve consistente una caché: si Redis conserva datos durante una falla de red, puede quedar una copia vieja hasta que caduque. El TTL y la lectura de PostgreSQL para autorizar siguen siendo necesarios. Incluso el borrado poscommit no elimina todas las carreras entre lectores y escritores concurrentes. En este lab la caché se admite desactualizada para mostrar, nunca para decidir.

Preguntas de cierre

1. ¿Por qué MongoDB para avisos y no para el saldo?
Vi que SMS, correo y push tienen campos distintos y se leen como documentos enteros por CLABE. Mongo permite guardar esa forma. El dinero exige controles transaccionales y de concurrencia entre cargos, abonos e idempotencia; en esta arquitectura esa verdad ya está en PostgreSQL y no la cambié. No significa que Mongo carezca de transacciones; se trata de no mover un dato crítico sin una razón y un diseño equivalente.

2. ¿Qué compré con la caché?
Medí 99 % de hit ratio en A y 83.33 % en B. Al cambiar saldos se borran copias y vuelven los misses. Con caché hubo una lectura a PostgreSQL por cada cien GET; sin caché hubo cien. Las medianas estuvieron cerca, alrededor de 4.55-5.37 ms con caché y 5.50-5.60 ms sin ella. Compré principalmente capacidad para dejar la base libre para escribir.

3. ¿Y si el cargo autorizara desde la caché?
En la Parte 4 habría visto 716 donde realmente había 5. Podría aceptar un gasto sin fondos si no hiciera una validación correcta en PostgreSQL. La pantalla puede atrasarse; la autorización no debe usar esa copia.

4. ¿Por qué DEL y después del commit?
Borrar obliga a que la próxima lectura reconstruya la copia desde la verdad. Hacer SET con una respuesta de una operación puede pisar un saldo más reciente que publicó otra operación. Si borro antes del commit, otro lector puede recargar el saldo anterior mientras el cambio aún no es visible. El borrado posterior reduce esa ventana; el TTL limita el daño si falla la invalidación.

Verificación. Pasaron 13 pruebas de cuenta y 20 de notificación. Actualicé las pruebas del consumidor para comprobar Mongo antes del ack y agregué casos de documentos, duplicados parciales, límite de diez y DEL diferido con Redis caído. Las pruebas de integración de esta bitácora se ejecutaron con los tres contenedores de cuenta.

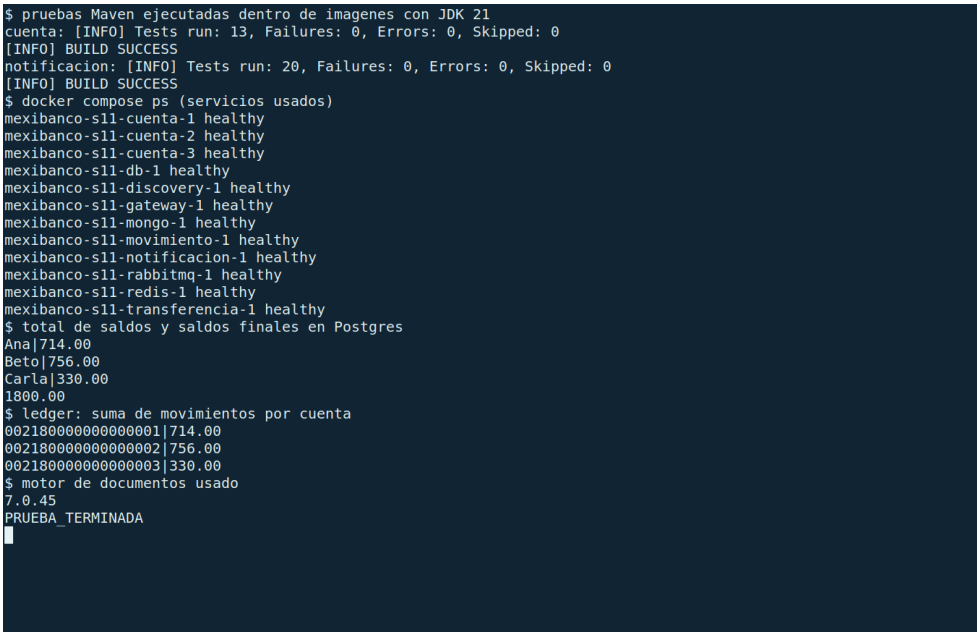
Entorno, código y fuentes

Leí los tres archivos del directorio s11-poliglota: README, guía HTML y guía PDF. Trabajé en una copia nueva de Mexi Banco, con proyecto, red y volúmenes del S11. SPEI quedó apagado porque no participa en estas pruebas. No borré volúmenes de otros laboratorios.

Adaptación local. MongoDB 8 no arrancó con el kernel de esta máquina por el problema SERVER-121912. Usé **mongo:7.0**, compatible con las inserciones, documentos, índices y explain del lab. Por eso entrego también docker-compose.yml. Ese archivo usa nombre/red propios y el tiempo de espera HTTP ajustado para el reto.

Archivos entregados. Las carpetas completas cuenta y notificacion, sin target; Compose modificado y esta bitácora. Los PNG originales y los scripts de evidencia quedan en /home/ivan/Documents/FI/distribuidos/s11-work; los logs y las cien muestras de cada corrida, en /home/ivan/Documents/FI/distribuidos/logs/s11.

Fuentes:
1. Material local del Lab S11 y rama 06-poliglota-base de github.com/OscarRuiz21/mexi-banco.
2. Spring Data Redis 3.3.4, RedisCacheWriter y TransactionAwareCacheDecorator (código oficial): github.com/spring-projects/spring-data-redis y github.com/spring-projects/spring-framework.
3. MongoDB, notas oficiales de versión e incompatibilidad de kernel: mongodb.com/docs/manual/release-notes/8.0.



Captura de pantalla. Pruebas automatizadas y comprobación final del entorno.